If no match is found, the entire loop is tried again from the receiver's class, this time for looking for a special method called method_missing. If no method_missing is found to handle the message, a NameError is thrown.

Class or module methods have a slightly different path:

- Methods added directly to the class or module via the def self.foo or class << self syntax.
- Methods in modules that are added to the receiving class or module via extend

And so on upward via superclasses.

Super Lookup

When executing a method, if Ruby encounters the keyword super, it acts as though a method of the same name as the method being executed had been called, but starts the search later. Specifically, the method lookup for super starts after the point where the method being executed is located. If the method being executed is defined inside the class as a normal instance method—step 3 in the above lookup loop—then Ruby starts looking in step 4, with included modules, then goes looking to the superclass. If, instead, the method is defined in a prepended module—step 1 in the above lookup loop—then Ruby starts looking in step 2, and step 3 for regular instance methods.

If super has no argument list, the same method arguments from the original method call are passed forward, if any argument list (even an empty one with empty parentheses) is specified, those arguments are passed through.

Short examples of method lookup are kind of contrived, but let's try one anyway:

```
module Log
  def execute
    puts "logging"
    super
  end
end

module Caller
  def execute
    puts "calling"
    super
  end
end

class Parent
  def execute
    puts "parenting"
  end
end

class Child < Parent
  prepend Log
  include Caller

  def execute
    puts "childing"
    super
  end
end
```